ATHANASIOS GOURDOMICHALIS

Numerical Analysis Python Programs
Computational Cost Comparison: Newton-Raphson vs. Secant Method

This document provides:

- A brief theoretical intro to the computational cost of the Newton-Raphson and Secant methods
- The problem statement and benchmark settings
- The code implementation details, results table, and convergence analysis.

1. Introduction to Computational Cost

Let us assume that **we want to find** the root x^* of a **function** $f(x)$ with a **specific tolerance** `tol`, such that $|x_n - x^*| \leq$ `tol`.

- **Computational Cost per Iteration (k flops):**
  - If we assume that **calculating the derivative** $f'(x)$ takes **about the same floating-point operations** (flops), denoted as k, as calculating $f(x)$, then the **Newton-Raphson (N-R) method** requires **two function evaluations per step**: $f(x_n)$ and $f'(x_n)$. Therefore, **its cost per iteration is**:

$$\text{Cost}_{\text{N-R}} = 2 * \text{k flops}$$

  - On the other hand, the **Secant method** does **not calculate the derivative directly**. Instead, **it approximates the derivative by using the two most recent points** and **reuses the previous function value**. This means **it only needs one new evaluation** of $f(x)$ per iteration:

$$\text{Cost}_{\text{Secant}} = 1 * \text{k flops}$$

- **Comparison of Convergence and Efficiency:**
  - **The order of convergence** of the **N-R method** is quadratic ($p = 2$), while the **Secant method** has **superlinear convergence** ($p \approx 1.618$).

- Because $\mathbf{2^2 = 4 \approx 4.23 = 1.618^3}$, **3 iterations of the Secant method** reduce **the error** about **as much as 2 iterations of the N-R method**.
- Comparing **the total cost** for **the same error reduction**:

$$\textbf{Cost of 2 N-R iterations} = 2 \times 2\text{k} = \mathbf{4k}$$

$$\textbf{Cost of 3 Secant iterations} = 3 \times 1\text{k} = \mathbf{3k}$$

- Therefore, since $\mathbf{4k > 3k}$, and **assuming that** evaluating $\boldsymbol{f'(x)}$ costs the same as $\boldsymbol{f(x)}$, the **Secant method is slightly faster in total computational cost** than the **N-R method**.

2. Problem Statement

- **Target Function:**

$$\boldsymbol{f(x) = x^4 + 2x^3 - 7x^2 + 3 = 0}$$

- **First derivative:**

$$\boldsymbol{f'^{(x)} = 4x^3 + 6x^2 - 14x}$$

- **Target Root and Tolerance:** We find the **positive root** $\boldsymbol{x^* \approx 1.6180}$ with **tolerance** $\mathbf{tol = 10^{-4}}$ and **maximum iterations** $\mathbf{maxit = 15}$.
- **Initial Approximations:**
  - **Newton-Raphson:** $\boldsymbol{x_0 = 1.4}$
  - **Secant:** $\boldsymbol{x_0 = 1.4,\ x_1 = 1.5}$
- **Comparison Metrics:**

1. **Total iterations needed to reach convergence.**
2. **Total number of function evaluations** ($\boldsymbol{f}$ and $\boldsymbol{f'}$).
3. **Theoretical cost** in **flops** ($\boldsymbol{10k}$ vs. $\boldsymbol{7k}$).
4. **Average CPU execution time** in **microseconds** ($\boldsymbol{\mu s}$) using the **`timeit` library** over $\mathbf{20,000}$ **runs**.

3. Code Description & Implementation Details

- **Function Evaluation Counters:**
  - For **Newton-Raphson**, we record **both** $f(x_0)$ and $f'(x_0)$ at **every step** $(2 * k$ **per iteration**).
  - For the **Secant method**, we **evaluate 2 points initially** $[f(x_0), f(x_1)]$ and then **only 1 new evaluation per step** $(1 * k$ **per iteration**) by **reusing the previous value**.
- **Benchmarking with `timeit`:**
  - We run **each method 20,000** times **using `timeit.timeit`** to **find the average time** in **microseconds** and **remove small computer lags**.

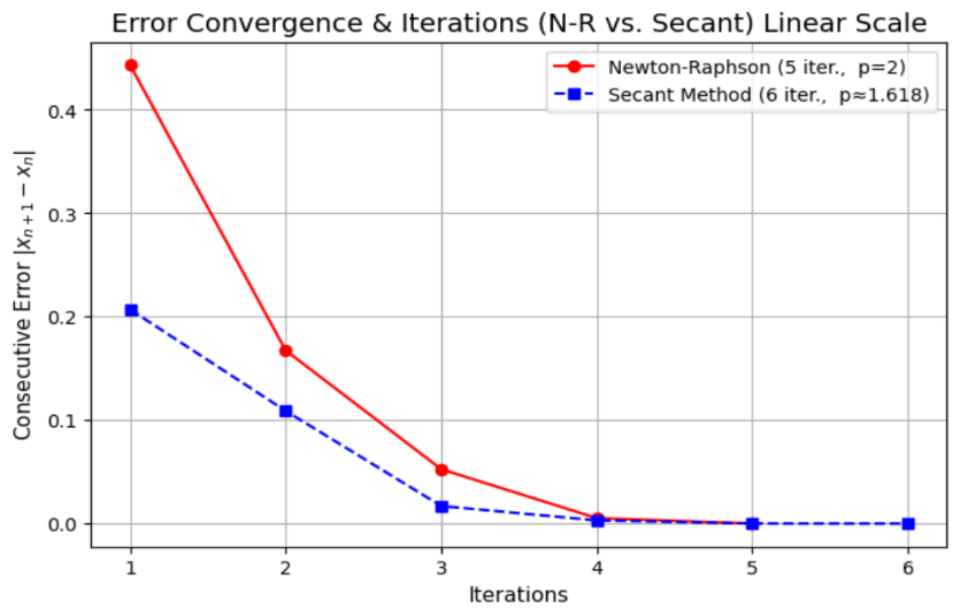
- **Summary Table of Results:**

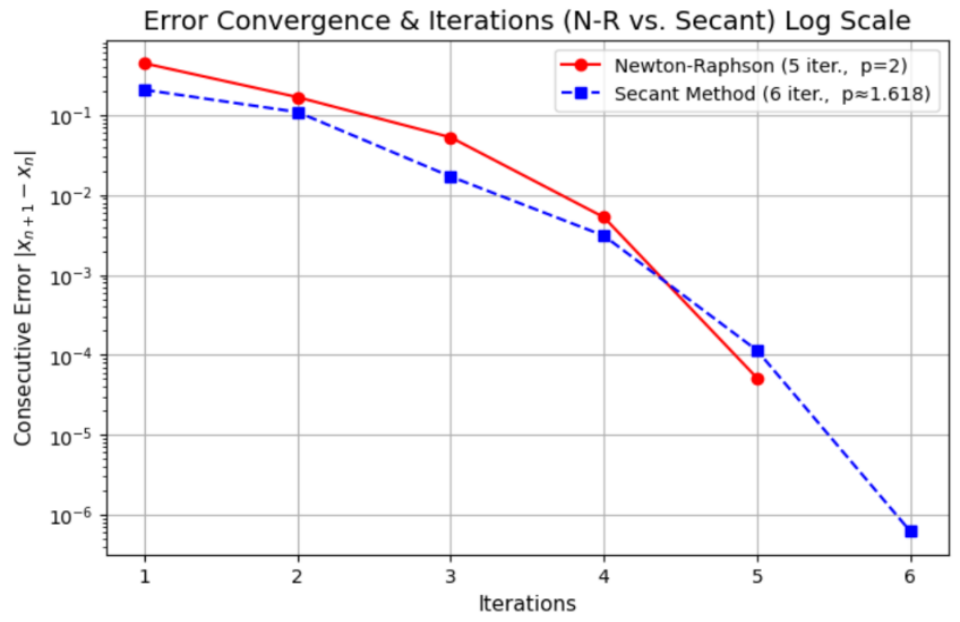
Metric	Newton-Raphson (N-R)	Secant Method
Initial Estimate(s)	$x_0 = 1.4$	$x_0 = 1.4, x_1 = 1.5$
Computed Root (x*)	1.6180	1.6180
Iterations to Converge	5	6
Function Evaluations (f + f')	10	7
Theoretical flops Cost	10k	7k
Final Error (`e_last`)	5.1418×10^{-5} (0.00005)	6.2415×10^{-7} (0.0000006)
Execution Time (`timeit`)	**20.83 µs**	**18.84 µs**

- **Plot Analysis and Conclusion:**

  - **Error vs. Iterations (Plots 1 & 3):**

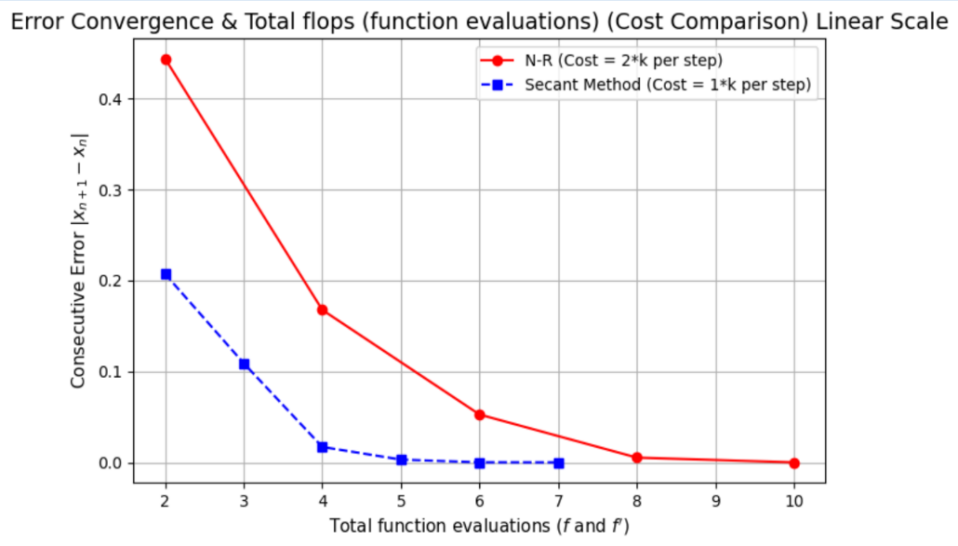
    In both **linear** and **log scales**, the **Newton-Raphson method** takes **fewer iterations** (**5** vs. **6**) because **it has a higher convergence rate** ($\boldsymbol{p = 2}$ vs. $\boldsymbol{p \approx 1.618}$).

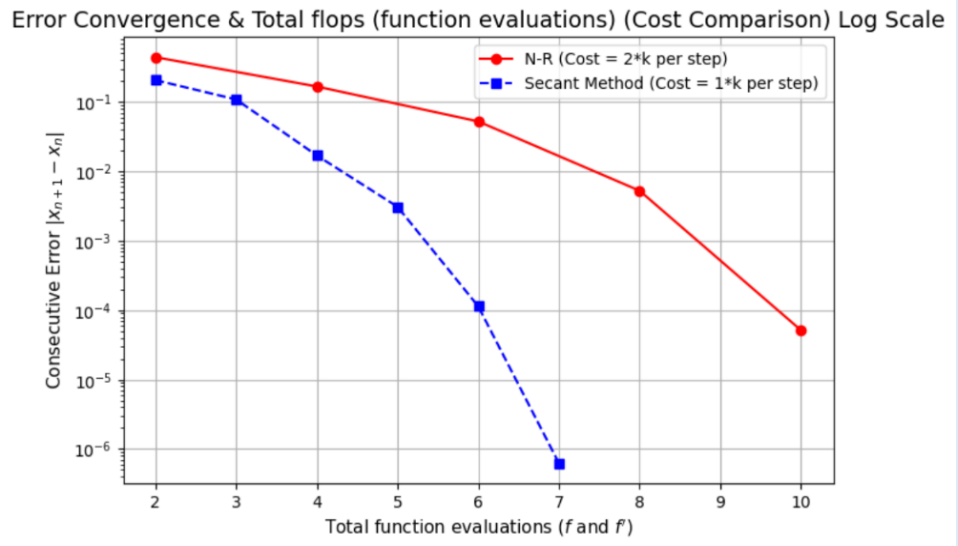




- **Error vs. Total Function Evaluations (Plots 2 & 4):**

  When **comparing the error** against **the total function evaluations**, the **Secant method converges faster**.
  The Secant **method reaches the tolerance in only 7 evaluations** $(7k)$, whereas Newton-Raphson needs **10 evaluations** $(10k)$.





- **Theoretical Confirmation:**

  This **lower evaluation cost** is **clearly reflected** in the **actual CPU time** $(18.84\ \mu s < 20.83\ \mu s)$.

The experiment completely confirms the theory: **even though the Secant method needs one more iteration**, its **cheaper cost per step** makes it **more efficient overall**
($\boldsymbol{7k < 10k}$ and $\boldsymbol{3k < 4k}$).